



CIA-3 PROJECT REPORT

Advanced JavaScript Backend Frameworks (Node.js & Express JS)

Field	Detail
Project Code & Title	P14 — Customer Support Helpdesk & Ticketing System
Course Name	Advanced JavaScript Backend Frameworks (Node.js & Express JS)
Semester	5th Semester
Department	ADSE
Section / Batch	5 BTCS AIML A
Institution	Christ University
Submission	September 12, 2026

Team Details

S.No	Student Name	Register No.	Department
1	George Thomas	2462072	ADSE
2	Femi K E	2462069	ADSE
3	Gokulakrishnan	2462075	ADSE
4	Gutha Nihitha	2463021	ADSE

GitHub Repository

Repository Link: https://github.com/Nihitha47/Customer_Support_Helpdesk-Ticketing_System

The repository contains the complete Node.js + Express.js + MongoDB (Mongoose) backend, a companion React (Vite + Tailwind) frontend under /frontend, and the project was verified through manual, module-by-module testing in Postman against both the local server and the Render-hosted deployment.

1. Project Overview

The Customer Support Helpdesk & Ticketing System (P14) is a role-based support-desk backend built for the CIA-3 assessment. Customers register, raise tickets against a manager-curated set of categories, track SLA deadlines, hold a public reply conversation with the assigned agent, escalate unresolved issues, and rate their experience once a ticket is closed. Agents work an assigned queue, update ticket status through a controlled lifecycle, and keep internal notes that stay hidden from the customer. Managers assign tickets, review and act on escalations, manage the category list, and monitor SLA compliance, escalation load, and CSAT scores through a dedicated analytics module. The system was developed by a four-member team, each owning a distinct set of modules, later merged into a single unified Express application with a companion React dashboard.

2. Problem Statement

Support organisations need a ticketing backend that enforces response-time discipline and keeps a defensible record of every status change — properties a plain CRUD ticket table does not provide. A naive implementation lets any authenticated user jump a ticket from Open straight to Closed, loses track of which agent is accountable once a ticket is escalated, and gives no way to tell whether a resolution happened inside its promised SLA window. This project addresses that gap by modelling an explicit ticket status lifecycle with a whitelist of legal transitions, computing a per-priority SLA deadline at creation (and recalculating it whenever priority changes), maintaining an append-only escalation history rather than a single overwritable status field, and separating the customer-visible reply thread from agent/manager-only internal notes.

3. Objectives

- Model users, tickets, categories, comments, internal notes, and satisfaction ratings in MongoDB with clear ownership and access rules.
- Enforce a controlled ticket status lifecycle (Open → In Progress → Resolved → Closed) instead of accepting arbitrary status writes.
- Compute and recalculate SLA deadlines per priority level, and detect SLA breaches automatically wherever a ticket is read or resolved.
- Implement a multi-level escalation workflow with an immutable, append-only audit history for every escalation event.
- Keep a customer-visible public reply thread strictly separate from agent/manager-only internal notes.
- Enforce JWT-based authentication and role-based access control across Customer, Agent, and Manager roles.
- Provide manager-facing analytics (agent workload and organisation-wide reporting) and a customer satisfaction (CSAT) feedback loop.
- Produce a GitHub-hosted codebase with a companion React frontend, verified end-to-end through Postman against a live Render deployment.

4. Actors & User Roles

Role	Responsibility
Customer	Registers, raises tickets against an active category, replies on the public thread, escalates a stalled ticket, and rates the resolution once it is Resolved or Closed.
Agent	Works the tickets assigned by a manager, replies to customers, keeps internal notes, and moves a ticket through the status lifecycle up to Resolved.
Manager	Assigns tickets to agents, changes priority, reviews and resolves escalations, manages the category catalogue, and monitors SLA/CSAT analytics; has the same operational access as an Agent plus administrative scope.

Scope, Assumptions & Boundaries

- Notification channels (email/SMS on SLA breach, escalation, or reply) are out of scope; all updates surface only inside the portal.
- Authentication is a self-built JWT flow (bcryptjs-hashed passwords, 1-day token expiry); social login was not required and was not implemented.
- A companion React + Vite + Tailwind single-page frontend (public/customer/agent/manager views) is included alongside the REST API.
- SLA and escalation timers rely on server wall-clock time rather than a scheduled job; a breach is detected the next time a ticket is read, assigned a new priority, or resolved — not the instant its deadline elapses.
- The category field on a ticket is stored as a validated string cross-checked against the categories collection, rather than an ObjectId reference, once at least one category has been seeded.
- The four-member team split ownership by feature area (auth/RBAC/categories, ticket core/assignment, SLA/status/priority, escalation/comments/analytics) and merged onto a single Express server.

5. Technology Stack & Architecture

Layer	Technology
Runtime & Server	Node.js, Express.js 5
Database & ODM	MongoDB, Mongoose 9
Authentication & Security	JSON Web Tokens (JWT), bcryptjs password hashing
Validation	express-validator on every mutating route, enforced by a shared validate.js middleware
Frontend	React 19, React Router, Vite, Tailwind CSS, lucide-react (frontend/)
Deployment	Backend deployed on Render (customer-support-helpdesk-ticketing.onrender.com); verified with Postman

The codebase follows an MVC-style layout with routes/, controllers/, models/, middleware/, and utils/ kept in separate folders, and a fully independent frontend/ React application that talks to the API over fetch calls in src/services/api.js. Controllers parse the request, call the appropriate business logic, and shape the JSON response; SLA-deadline math lives in utils/sla.js so it can be reused by both ticket creation and priority updates. A single centralized error-handling middleware (middleware/errorHandler.js) returns a consistent { success, message } JSON shape for CastError, duplicate-key (11000), and default failures, so no unhandled rejection reaches the client as a raw stack trace.

6. Functional Modules (14 / 14 Implemented)

All fourteen modules below are implemented and demonstrably working against the deployed backend. Ownership was split by feature area (auth-rbac-categories, ticket-core-assignment, sla-status-priority, escalation-comments-analytics) and merged into a single unified server, with every member individually accountable for explaining modules outside their own ownership area during the viva.

#	Module	Owning Member	Code Location
1	Customer Registration & Login (JWT)	George Thomas	controllers/authController.js, models/User.js
2	Role-Based Access Control (RBAC)	George Thomas	middleware/auth.js, middleware/role.js
3	Category Management	George Thomas	controllers/categoryController.js, models/Category.js
4	Ticket Creation & Category Validation	Femi K E	controllers/ticketController.js, models/Ticket.js
5	Ticket Listing, Filtering, Search & Pagination	Femi K E	controllers/ticketController.js (getTickets)
6	Ticket Assignment to Agents	Femi K E	controllers/ticketController.js (assignTicket)
7	Ticket Status Lifecycle Management	Gokulakrishnan	controllers/ticketController.js (updateTicketStatus)
8	SLA Deadline Calculation & Breach Detection	Gokulakrishnan	utils/sla.js, controllers/slaController.js
9	Ticket Priority Management & SLA Recalculation	Gokulakrishnan	controllers/ticketController.js (updateTicketPriority)
10	Escalation Workflow (Trigger + Manager Review)	Gutha Nihitha	controllers/ticketController.js (escalateTicket, reviewEscalation)
11	Public Reply Thread (Comments)	Gutha Nihitha	controllers/commentController.js, models/Comment.js
12	Internal Notes (Agent / Manager only)	Gutha Nihitha	controllers/internalNoteController.js, models/InternalNote.js
13	Customer Satisfaction Rating (CSAT)	Gutha Nihitha	models/SatisfactionRating.js (ticketController.js)
14	Agent Workload & Manager Analytics Dashboard	Gutha Nihitha	controllers/analyticsController.js

Business-Rule & Workflow Highlights

The evaluation criteria weight status-transition and workflow logic over plain CRUD. The following are the non-trivial business-rule chains implemented across the modules:

- Ticket Status Lifecycle — status can only move through an explicit allowedTransitions map (Open → In Progress → Resolved → Closed); any other jump is rejected with a 400 and a message naming the illegal transition.
- SLA Engine — a per-priority SLA_HOURS table (Low: 48h, Medium: 24h, High: 8h, Urgent: 4h) fixes the deadline at creation time; changing priority while a ticket is Open or In Progress recalculates slaDeadline from the ticket's original createdAt rather than the current time.
- Escalation Workflow — a customer (for their own ticket), the assigned agent, or a manager can trigger an escalation with a reason; every trigger and every manager review appends a new entry to an escalationHistory array rather than overwriting the previous one, producing a queryable audit trail.
- Manager Escalation Review — a single endpoint lets a manager reassign the agent, change priority (which recalculates the SLA deadline), and set the escalation status (Under Review / Escalated / Resolved) in one call, with the outcome recorded in escalationHistory.
- Internal Notes vs. Public Comments — the reply thread (Comment) is visible to the ticket's customer, its assigned agent, and any manager, while InternalNote is restricted to the assigned agent or a manager only, enforced by a dedicated canAccessNotes() check.
- Customer Satisfaction Rating — a rating can only be submitted once a ticket is Resolved or Closed, is limited to one submission per ticket (duplicate check against SatisfactionRating), and must be an integer between 1 and 5.
- Category-Validated Ticket Creation — once at least one category exists in the catalogue, createTicket rejects any ticket whose category name does not match an active category, preventing free-text category drift.

7. Database Architecture

Child collections reference their parent by ObjectId (customerId, assignedAgentId, ticketId) rather than embedding, since comments, internal notes, and satisfaction ratings are created and queried independently of the ticket document. The one deliberate exception is ticket.category, which is stored as a plain, validated string rather than an ObjectId reference to categories — kept simple because a ticket's category is a point-in-time label, not a live join target.

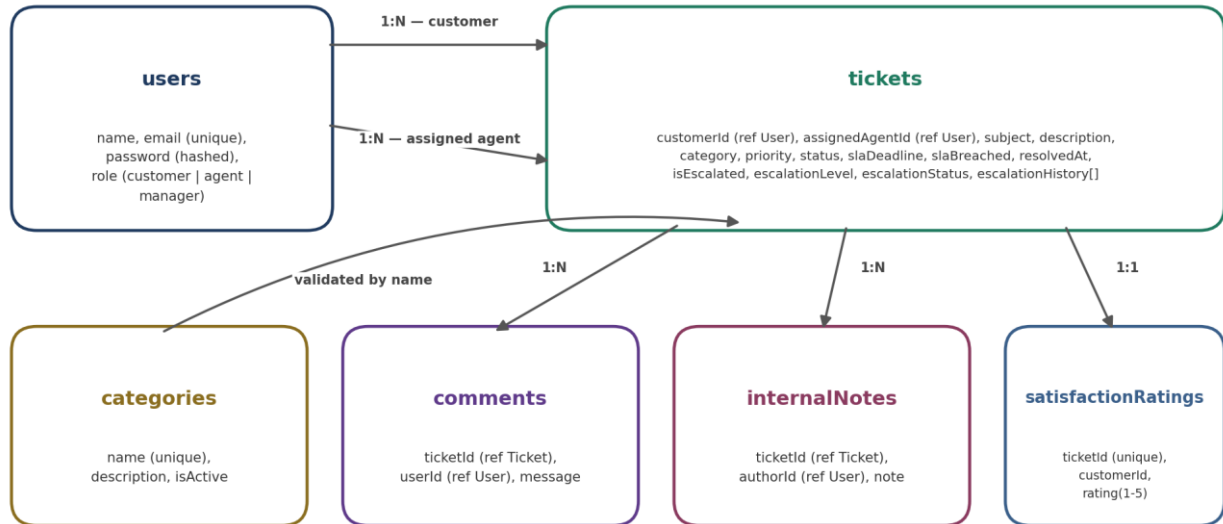
Collections & Key Fields

Collection	Key Fields
users	name, email (unique), password (bcrypt hash), role (customer agent manager)
categories	name (unique), description, isActive
tickets	customerId (ref User), assignedAgentId (ref User), subject, description, category, priority (Low Medium High Urgent), status (Open In Progress Resolved Closed), slaDeadline, slaBreached, slaBreachedAt, resolvedAt, isEscalated, escalationLevel, escalationStatus, escalationReason, escalatedAt, escalatedBy, escalationHistory[]
comments	ticketId (ref Ticket), userId (ref User), message
internalnotes	ticketId (ref Ticket), authorId (ref User), note
satisfactionratings	ticketId (ref Ticket, unique), customerId (ref User), rating (1–5), feedback

Indexes

Collection	Index	Reason
users	{ email: 1 } (unique)	Enforces one account per email and speeds up login lookups.
categories	{ isActive: 1 }	Speeds up fetching the active category list shown to customers.
tickets	{ customerId: 1 }, { assignedAgentId: 1 }, { status: 1 }, { priority: 1 }, { category: 1 }, { isEscalated: 1 }, { slaBreached: 1 }	Speeds up role-scoped ticket queries, dashboard filters, and SLA/escalation monitoring.
comments	{ ticketId: 1 }	Speeds up loading a ticket's public reply thread.
internalnotes	{ ticketId: 1 }	Speeds up loading a ticket's internal notes.
satisfactionratings	{ customerId: 1 }, { rating: 1 }, { ticketId: 1 } (unique)	Speeds up CSAT aggregation and blocks duplicate ratings per ticket.

Entity-Relationship Diagram



*All child collections reference their parent by ObjectId (customerId / assignedAgentId / ticketId).
 ticket.category is stored as a validated string cross-checked against categories.name rather than an ObjectId reference.*

Both relationships between users and tickets (as customer and as assigned agent) are one-to-many, modelled with a userId-style ObjectId field on the ticket rather than an embedded array on the user, since a customer's or agent's ticket list is large and queried independently. comments, internalnotes, and satisfactionratings each carry a ticketId reference back to their parent ticket; satisfactionratings additionally enforces a unique index on ticketId to guarantee at most one rating per ticket.

8. Complete REST API Endpoint Reference

Authentication (Module 1)

Method	Endpoint	Auth / Role	Description
POST	/api/auth/register	Public	Register a new customer account
POST	/api/auth/login	Public	Authenticate a user and return a signed JWT
GET	/api/auth/me	Authenticated	Fetch the authenticated user's own profile
GET	/api/auth/agents	AGENT, MANAGER	List all support agents (for ticket assignment)
POST	/api/auth/seed	Public	Development helper: seed default categories and demo accounts

Category Management (Module 3)

Method	Endpoint	Auth / Role	Description
GET	/api/categories	Public / Authenticated	List active categories (managers may pass includeInactive=true)
POST	/api/categories	MANAGER	Create a new ticket category
PUT	/api/categories/:id	MANAGER	Update a category's name, description, or active flag
PATCH	/api/categories/:id/status	MANAGER	Toggle a category's active status

Ticket Management (Modules 4–10, 13)

Method	Endpoint	Auth / Role	Description
GET	/api/tickets	Authenticated	List tickets with role-based scoping, filters, search, sort & pagination
POST	/api/tickets	Authenticated (customer)	Create a new ticket (category-validated, SLA deadline computed)
GET	/api/tickets/:id	Authenticated	Get ticket details; SLA breach is checked and persisted on read
GET	/api/tickets/:id/sla	Authenticated	Check a ticket's SLA deadline and breach status
PUT	/api/tickets/:id/assign	MANAGER	Assign a ticket to an agent (moves status to In Progress)
PUT	/api/tickets/:id/status	AGENT, MANAGER	Update ticket status through the controlled lifecycle
PUT	/api/tickets/:id/priority	MANAGER	Update priority and recalculate the SLA deadline
POST	/api/tickets/:id/escalate	Authenticated	Escalate a ticket with a reason (customer/agent/manager)
PUT	/api/tickets/:id/escalation-review	MANAGER	Review an escalation: reassign, reprioritise, set status
POST	/api/tickets/:id/comments	Authenticated	Add a public reply to the ticket's conversation thread
GET	/api/tickets/:id/comments	Authenticated	Get the public reply thread
POST	/api/tickets/:id/notes	AGENT, MANAGER	Add a staff-only internal note
GET	/api/tickets/:id/notes	AGENT, MANAGER	Get staff-only internal notes
POST	/api/tickets/:id/satisfaction	CUSTOMER	Submit a 1–5 CSAT rating (Resolved/Closed tickets only)
GET	/api/tickets/:id/satisfaction	Authenticated	Get the CSAT rating recorded for a ticket

Analytics & Utility (Module 14)

Method	Endpoint	Auth / Role	Description
GET	/api/analytics/agent-workload	AGENT, MANAGER	Per-agent workload: active load, SLA breaches, escalations, avg. resolution time
GET	/api/analytics/manager-reports	MANAGER	Organisation-wide status, priority, SLA, escalation & CSAT report
GET	/api/health	Public	API health check
GET	/api/protected	Authenticated	Protected test route (legacy, preserved from initial scaffold)

9. Sample Request & Response (captured via Postman)

The requests and responses below are reproduced exactly as captured against the deployed backend at <https://customer-support-helpdesk-ticketing.onrender.com>, without modification.

Customer Registration — POST /api/auth/register

```
REQUEST BODY
{
  "name": "Postman Customer",
  "email": "postman.customer@example.com",
  "password": "Customer@123"
}

RESPONSE • 201 Created
{
  "success": true,
  "message": "User registered successfully",
  "user": {
    "id": "6aa24892e67d1e28398cba2f",
    "name": "Postman Customer",
    "email": "postman.customer@example.com",
    "role": "customer"
  }
}
```

Customer Login — POST /api/auth/login

```
REQUEST BODY
{
  "email": "postman.customer@example.com",
  "password": "Customer@123"
}

RESPONSE • 200 OK
{
  "success": true,
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhYTI0ODkyZTY3ZDF1MjgzOThjYmEyZiIsInJvbGUiOiJjdXN0b211ciJ9.XMaHW6urPt3TAvoc-wFWIpDADwGDjARKjrv7TG61Y0A",
  "user": {
    "id": "6aa24892e67d1e28398cba2f",
    "name": "Postman Customer",
    "email": "postman.customer@example.com",
    "role": "customer"
  }
}
```

Authenticated Profile — GET /api/auth/me

```
RESPONSE • 200 OK
{
  "success": true,
  "user": {
```

```
"id": "6aa24892e67d1e28398cba2f",
"name": "Postman Customer",
"email": "postman.customer@example.com",
"role": "customer"
}
}
```

Ticket Creation — POST /api/tickets

REQUEST BODY

```
{
  "subject": "Postman Ticket Test",
  "description": "Testing ticket creation through the deployed API.",
  "category": "Account Access",
  "priority": "High"
}
```

RESPONSE • 201 Created

```
{
  "success": true,
  "message": "Ticket created successfully",
  "ticket": {
    "customerId": "6aa24892e67d1e28398cba2f",
    "subject": "Postman Ticket Test",
    "description": "Testing ticket creation through the deployed API.",
    "category": "Account Access",
    "priority": "High",
    "status": "Open",
    "assignedAgentId": null,
    "slaDeadline": "2026-09-10T14:09:41.264Z",
    "slaBreached": false,
    "slaBreachedAt": null,
    "resolvedAt": null,
    "isEscalated": false,
    "escalationLevel": 0,
    "escalationStatus": "None",
    "escalationReason": null,
    ... (as captured; response continues beyond this point)
  }
}
```

10. Non-Functional Requirements Implementation

Requirement	How It Is Met
Security	Passwords hashed with bcryptjs; every protected route passes through JWT-verification middleware (protect) before reaching a controller.
Validation	Every mutating request body is validated with express-validator via a shared validate.js middleware before it reaches business logic.
Error Handling	A single centralized Express error-handling middleware returns a consistent { success, message } JSON shape for CastError, duplicate-key, and default failures.
Role-Based Access	authorizeRoles() middleware gates manager/agent-only routes; per-ticket ownership checks (customerId / assignedAgentId comparisons) additionally gate ticket detail, comments, notes, and ratings.
Code Structure	MVC-style layout: routes/, controllers/, models/, middleware/, utils/, with an independent frontend/ React application.
API Documentation	Every endpoint is documented in this report; representative flows were exercised and captured directly in Postman (Section 9 and Section 16).

11. Testing & Postman Verification

Correctness was verified by exercising representative flows directly against both the local server and the Render-hosted deployment (<https://customer-support-helpdesk-ticketing.onrender.com/api>) from Postman, rather than through an automated Jest/Supertest suite. The verified flows include: customer registration, login and Bearer-token retrieval of the authenticated profile (/api/auth/me), and category-validated ticket creation with server-computed SLA deadlines. The captured request/response pairs for these calls are reproduced unmodified in Section 9, and the corresponding Postman screens are included as screenshots in Section 16.

The web application itself was independently walked through end-to-end using its three quick-role logins (Customer, Agent, Manager): a customer ticket was raised from the Submit Ticket form, appeared on the manager's All Tickets board with a live SLA countdown, and the same board separately confirms the escalation workflow, showing a ticket in the URGENT / In Progress / Breach Imminent-Passed / Escalated state.

12. Common Implementation Pitfalls — How They Were Avoided

Pitfall	How the Codebase Avoids It
Storing plain-text passwords or secrets in source	Passwords are hashed with bcryptjs (10 salt rounds); JWT_SECRET and MONGO_URI are read from process.env via dotenv, and .env is listed in .gitignore.
Relying on frontend-only validation	Every mutating endpoint (register, login, ticket create/status/priority/escalate, comments, notes, ratings) validates its body server-side with express-validator, independent of the React frontend.
Any authenticated user calling every route	authorizeRoles() restricts manager/agent-only endpoints, and explicit ownership checks (ticket.customerId / ticket.assignedAgentId vs. req.user.id) gate ticket detail, comments, notes, and ratings.
Unhandled rejections crashing the server	Every controller wraps its logic in try/catch and forwards errors to next(error); a centralized errorHandler.js returns a clean JSON error response instead of a stack trace.
Free-text category drift on ticket creation	createTicket cross-checks the submitted category name against the categories collection once any category has been seeded, rejecting inactive or unknown categories.
Skipping ticket lifecycle stages	updateTicketStatus enforces an explicit allowedTransitions map (Open → In Progress → Resolved → Closed) instead of accepting any status value in the request body.

13. Known Limitations & Scope Boundaries

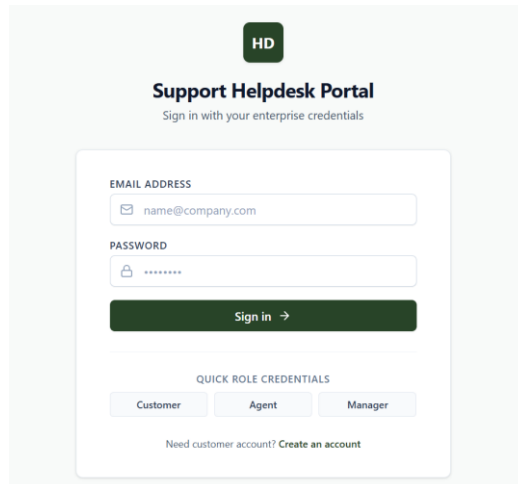
- ticket.category is a validated string cross-checked against categories.name at write time rather than a true ObjectId reference, so renaming a category later does not retroactively update tickets already filed against the old name.
- SLA and escalation breach detection rely on server wall-clock time evaluated on read/write (no dedicated cron or queue job), so a breach is flagged the next time a ticket is opened or updated rather than the instant its deadline passes.
- No automated Jest/Supertest suite is included in the repository at the time of this report; correctness was verified through manual, module-by-module Postman testing (Section 9, Section 16).
- Email/SMS notifications for replies, SLA breaches, and escalations are not implemented; all updates are visible only inside the customer/agent/manager portal.
- A single JWT secret with a fixed 1-day expiry is used; there is no refresh-token or explicit token-revocation mechanism.

14. Risks & Mitigation (as Executed)

Risk	Mitigation Applied
Merge conflicts from four members working in parallel	Ownership was split by feature-area boundary (auth/RBAC/categories, ticket core/assignment, SLA/status/priority, escalation/comments/analytics) against a shared Mongoose schema agreed early.
Category name drift breaking ticket creation	Category creation and status toggling are manager-only, and createTicket validates the submitted category name against the live categories collection before saving.
SLA breaches going unnoticed until a ticket is opened	The breach flag is recomputed on every GET /api/tickets/:id and persisted immediately, so any user opening a ticket refreshes its true SLA state.
Escalation history being silently overwritten	escalationHistory is implemented as an append-only embedded array (not a single mutable field), so every trigger and every manager review adds a new, timestamped entry.
Environment differences between local and hosted deployment	Connection string and JWT secret are externalized via .env, and the /api/auth/seed endpoint provisions the same demo accounts (Support Manager, Sarah Jenkins, David Miller, Alex Robinson) on any environment.

16. Screenshots

Web Application



HD

Support Helpdesk Portal

Sign in with your enterprise credentials

EMAIL ADDRESS

name@company.com

PASSWORD

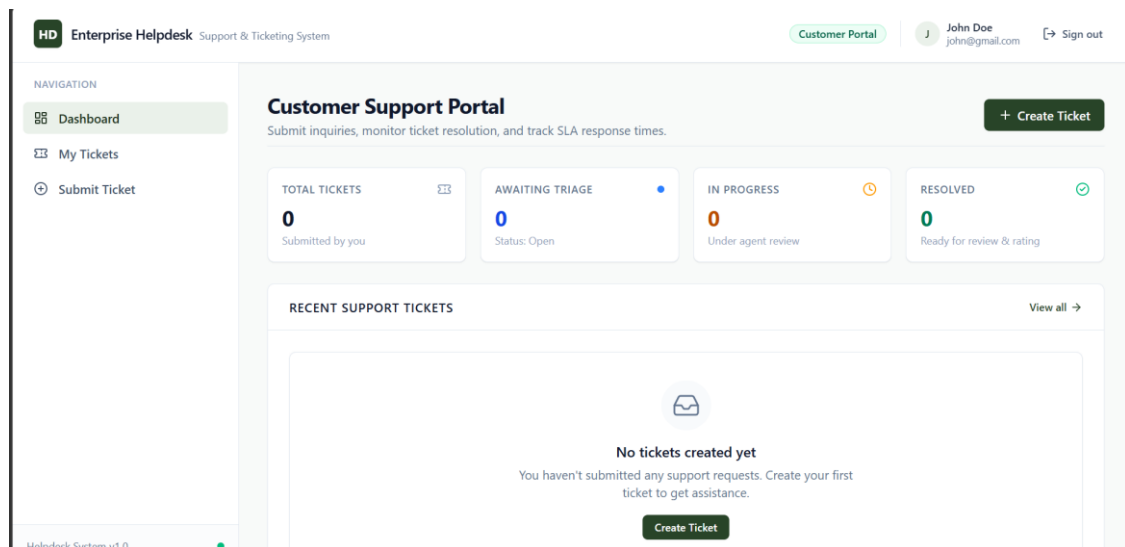
Sign in →

QUICK ROLE CREDENTIALS

Customer Agent Manager

Need customer account? [Create an account](#)

Support Helpdesk Portal — sign-in with quick role credentials (Customer / Agent / Manager)



HD Enterprise Helpdesk Support & Ticketing System

Customer Portal John Doe john@gmail.com Sign out

NAVIGATION

- Dashboard
- My Tickets
- Submit Ticket

Customer Support Portal

Submit inquiries, monitor ticket resolution, and track SLA response times.

+ Create Ticket

TOTAL TICKETS	AWAITING TRIAGE	IN PROGRESS	RESOLVED
0 Submitted by you	0 Status: Open	0 Under agent review	0 Ready for review & rating

RECENT SUPPORT TICKETS

View all →

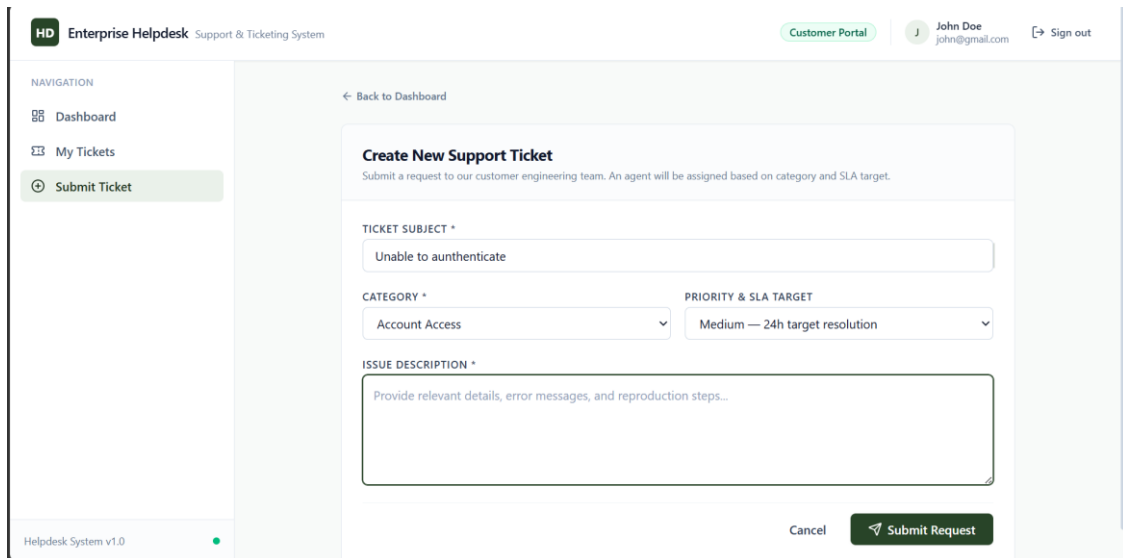
No tickets created yet

You haven't submitted any support requests. Create your first ticket to get assistance.

Create Ticket

Helpdesk System v1.0

Customer Support Portal — dashboard with ticket status summary



The screenshot displays the 'Create New Support Ticket' interface of the 'Enterprise Helpdesk Support & Ticketing System'. The top navigation bar includes the system name, a 'Customer Portal' link, and a user profile for 'John Doe' with a 'Sign out' option. A left sidebar contains navigation links: 'Dashboard', 'My Tickets', and 'Submit Ticket' (which is highlighted). The main content area features a 'Back to Dashboard' link and the form title 'Create New Support Ticket' with a sub-instruction: 'Submit a request to our customer engineering team. An agent will be assigned based on category and SLA target.' The form contains three main sections: 'TICKET SUBJECT *' with a text input field containing 'Unable to authenticate'; 'CATEGORY *' with a dropdown menu showing 'Account Access'; and 'PRIORITY & SLA TARGET' with a dropdown menu showing 'Medium — 24h target resolution'. Below these is the 'ISSUE DESCRIPTION *' section with a large text area containing the placeholder text 'Provide relevant details, error messages, and reproduction steps...'. At the bottom right of the form are 'Cancel' and 'Submit Request' buttons. The footer of the interface shows 'Helpdesk System v1.0'.

Create New Support Ticket — subject, category, priority/SLA target, description

HD

Enterprise Helpdesk

Support & Ticketing System

Customer Portal

J

John Doe

john@gmail.com

Sign out

NAVIGATION

Dashboard

My Tickets

Submit Ticket

Back to Tickets

#398cba30

Open

MEDIUM

23h 59m remaining

Request Escalation

Unable to authenticate

CATEGORY

Account Access

ASSIGNED SPECIALIST

Awaiting Agent Assignment

TARGET SLA DEADLINE

9/11/2026, 11:37:14 AM

SUBMITTED ON

9/10/2026, 11:37:14 AM

INITIAL ISSUE DESCRIPTION

Provide relevant errors

Support Conversation (0)

Public Reply Thread

No replies yet. Use the message box below to submit updates or additional details.

Customer ticket detail — SLA countdown and Request Escalation action

All Tickets

Agent Workload

Reports & Analytics

Category Settings

Operations Management

Support Manager

manager@helpdesk.com

Sign out

Search by subject or description...

All Priorities

All Categories

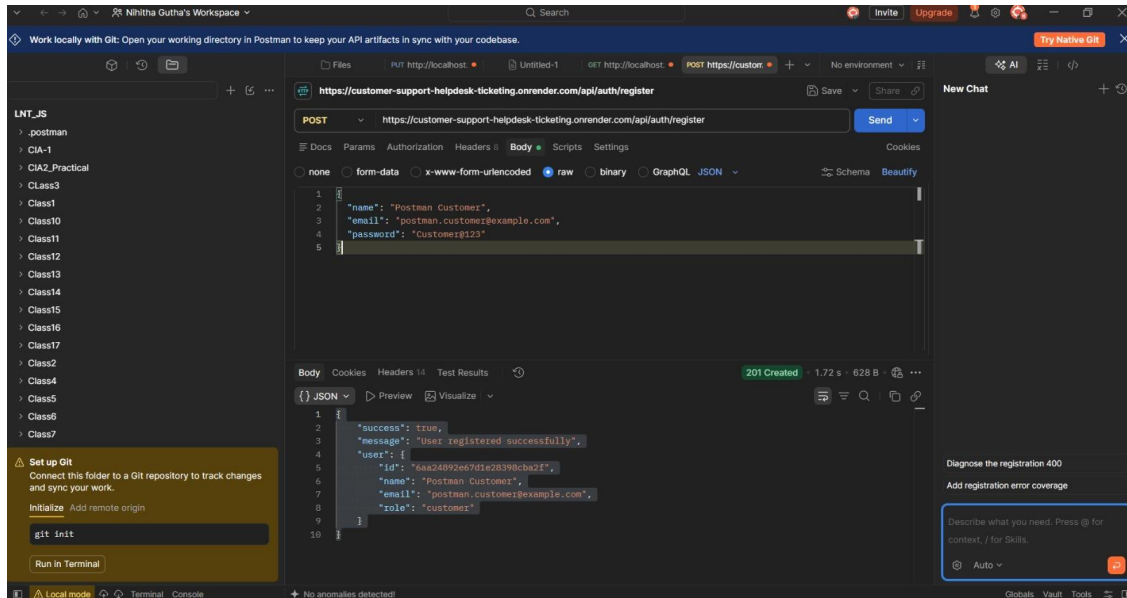
All Agents

SUBJECT	CUSTOMER	ASSIGNED	PRIORITY	STATUS	SLA	ESCALATION	ACTION
Unable to authenticate	John Doe	Unassigned	MEDIUM	Open	23h 59m remaining		Review
test ticket	Alex Robinson	Sarah Jenkins	LOW	Resolved	SLA Fulfilled		Review
Unable to access account	Alex Robinson	Sarah Jenkins	URGENT	In Progress	Breach Imminent / Passed	Escalated	Review
Test Integration Ticket	Alex Robinson	Sarah Jenkins	HIGH	Closed	SLA Fulfilled		Review
Agent authorization test	Test Agent	Unassigned	MEDIUM	Open	No SLA Set		Review
Unable to login	Test Customer	Test Agent	HIGH	Closed	SLA Fulfilled		Review

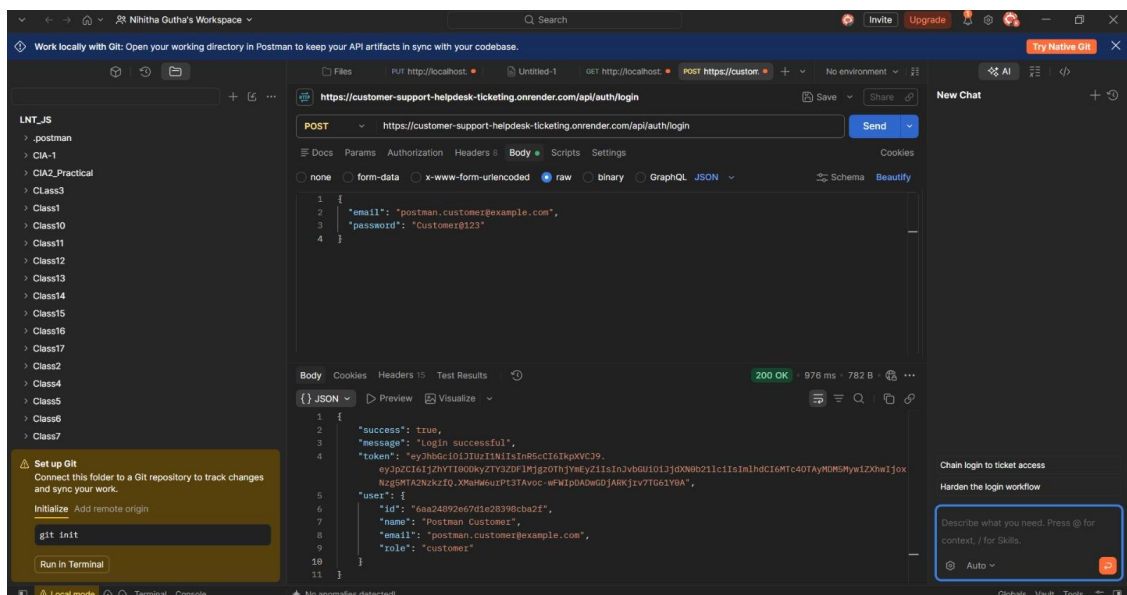
Helpdesk System v1.0

Manager — All Tickets board, showing an URGENT ticket in Breach Imminent/Passed and Escalated state

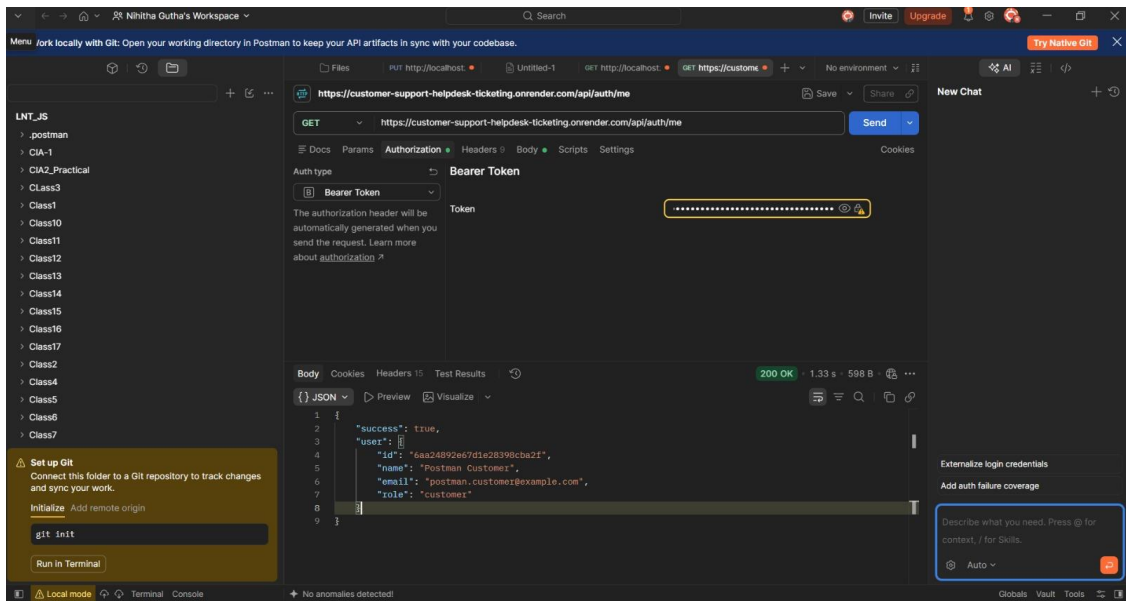
API Verification (Postman — deployed backend)



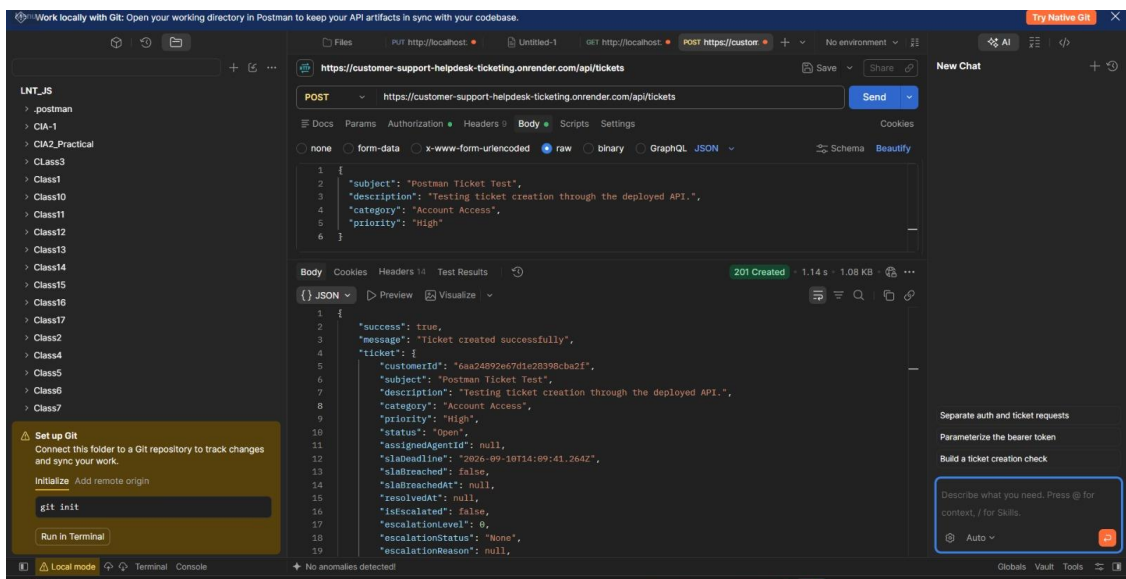
POST /api/auth/register — 201 Created



POST /api/auth/login — 200 OK, Bearer token issued



GET /api/auth/me — 200 OK, authenticated via Bearer token



POST /api/tickets — 201 Created, SLA deadline computed server-side

17. Conclusion & Learnings

Building this project as four independently-owned feature areas surfaced the same class of integration risk a single-developer build would not: keeping the escalation history genuinely append-only rather than an overwritable status field, making sure the SLA deadline was recalculated — not just re-read — whenever priority changed, and drawing a firm line between the customer-visible comment thread and the staff-only internal notes so neither controller could accidentally leak into the other's audience. The most valuable outcome was treating 'it returns 200' and 'it enforces the rule' as separate claims: exercising the deployed API directly from Postman — register, login, fetch profile, and create a ticket — and capturing the exact responses shown in Section 9 caught details (such as the server, not the client, deciding the SLA deadline) that reading the controller code alone would not have proven.